

Examen MU4IN505 CPA - Réponses

Haotian XUE

Exercice 1: Estimation du temps de calcul (2 points)

Q1

Estimer le temps de calcul du processus pour différentes valeurs de n avec une complexité $\Theta(n^2)$.

$$\begin{aligned}\text{Pour } n = 1K : \quad T &= \frac{(1K)^2}{1G} = \frac{(10^3)^2}{10^9} = 10^{-3} \text{ secondes} \\ \text{Pour } n = 10K : \quad T &= \frac{(10K)^2}{1G} = \frac{(10^4)^2}{10^9} = 10^{-1} \text{ secondes} \\ \text{Pour } n = 1M : \quad T &= \frac{(1M)^2}{1G} = \frac{(10^6)^2}{10^9} = 10^3 \text{ secondes} \\ \text{Pour } n = 1G : \quad T &= \frac{(1G)^2}{1G} = \frac{(10^9)^2}{10^9} = 10^9 \text{ secondes} \\ \text{Pour } n = 1T : \quad T &= \frac{(1T)^2}{1G} = \frac{(10^{12})^2}{10^9} = 10^{15} \text{ secondes}\end{aligned}$$

Q2

Donner les mêmes estimations en supposant une complexité $\Theta(n \log n)$.

$$\begin{aligned}
\text{Pour } n = 1K : \quad T &= \frac{1K \cdot \log 1K}{1G} = \frac{10^3 \cdot 10}{10^9} = 10^{-5} \text{ secondes} \\
\text{Pour } n = 10K : \quad T &= \frac{10K \cdot \log 10K}{1G} = \frac{10^4 \cdot 13.3}{10^9} = 1.33 \cdot 10^{-4} \text{ secondes} \\
\text{Pour } n = 1M : \quad T &= \frac{1M \cdot \log 1M}{1G} = \frac{10^6 \cdot 20}{10^9} = 2 \cdot 10^{-2} \text{ secondes} \\
\text{Pour } n = 1G : \quad T &= \frac{1G \cdot \log 1G}{1G} = \frac{10^9 \cdot 30}{10^9} = 30 \text{ secondes} \\
\text{Pour } n = 1T : \quad T &= \frac{1T \cdot \log 1T}{1G} = \frac{10^{12} \cdot 40}{10^9} = 4 \cdot 10^4 \text{ secondes}
\end{aligned}$$

Exercice 2: Enveloppe convexe (6 points)

Q1

Donner le code d'une fonction qui prend en entrée P un polygone simple comprenant n points et qui retourne en temps $O(n)$ l'enveloppe convexe de P .

La fonction peut être implémentée en utilisant le filtre de Graham. Les points du polygone étant déjà ordonnés en suivant le sens trigonométrique, nous devons juste vérifier les triplets de points successifs et enlever les points internes.

Q2

Donner le code d'une fonction qui prend en entrée deux polygones convexes P_1, P_2 comprenant au total n points et qui retourne en temps $O(n)$ l'enveloppe convexe de $P_1 \cup P_2$.

La fonction utilise l'algorithme de Jarvis pour fusionner les deux polygones convexes en temps linéaire.

Q3

Utiliser la précédente implémentation et donner le code d'une fonction récursive qui prend en entrée une liste quelconque de n points et qui retourne en temps $O(n \log n)$ l'enveloppe convexe de ces points.

La fonction utilise une approche de type diviser-pour-régner, similaire à l'algorithme TriFusion, pour calculer l'enveloppe convexe.

Q4

Donner le code d'une fonction qui prend en entrée une liste quelconque de n entiers et qui retourne en temps $O(n + \tau)$ ces entiers triés par ordre croissant, en utilisant une fonction `enveloppeConvexe` prenant en entrée n points dans le plan et retournant en temps τ l'enveloppe convexe de ces points.

La complexité optimale pour trier une liste de n entiers est $\Theta(n \log n)$. La complexité optimale pour calculer l'enveloppe convexe d'un ensemble de n points dans le plan est également $\Theta(n \log n)$.

Exercice 3: Tri paniers (6 points)

Q1

Donner un algorithme naïf résolvant la suppression de duplicata d'une liste de n entiers en temps $O(n^2)$.

L'algorithme parcourt chaque élément et vérifie s'il est déjà présent dans la sous-liste de résultats. Si oui, il est ignoré, sinon, il est ajouté.

Q2

Utiliser l'approche fonctionnelle pour résoudre la suppression de duplicata d'une liste L contenant n entiers en temps $O(n + X)$. Proposer une valeur de X .

Avec $X = \max(L)$, initialiser un tableau Panier de taille $X + 1$ et appliquer la fonction λ à chaque élément de L .

Q3

Utiliser une fonction de hachage pour résoudre la suppression de duplicata d'une liste de n entiers en temps $O(n + 2^{512})$.

La fonction de hachage λ_f permet d'accéder aux éléments en temps constant.

Q4

Expliquer par un dessin et par pseudo-code les étapes principales d'un algorithme de suppression de duplicata en temps quasi-linéaire en utilisant une structure de liste doublement chaînée.

L'algorithme utilise une liste doublement chaînée pour stocker les valeurs hachées, permettant un accès rapide et une suppression efficace des duplicata.

Q5

Appliquer les mêmes procédures pour trier une liste de n entiers en temps quasi-linéaire.

La complexité optimale du tri est $\Theta(n \log n)$. Les méthodes de suppression de duplicata ne peuvent pas être directement utilisées pour le tri en temps quasi-linéaire.

Exercice 4: Programmation dynamique (6 points)

Q1

Proposer un algorithme calculant M_{dist} en passant par le calcul de M_p , pour $2 \leq p \leq n$. Quelle est sa complexité?

$$M_{p+1}(i, j) = \min_k (M_p(i, k) + M_2(k, j))$$

Complexité: $O(n^4)$.

Q2

Donner une formule de récurrence pour le calcul de N_{p+1} à partir de la connaissance de N_p . En déduire un calcul plus rapide de M_{dist} en passant par le calcul de N_p . Quelle est sa complexité?

$$N_{p+1}(i, j) = \min(N_p(i, j), N_p(i, p+1) + N_p(p+1, j))$$

Complexité: $O(n^3)$.

Q3

Proposer un algorithme calculant la valeur S . Quelle est sa complexité?

$$M^{i+1}(w) = \max(M^i(w), v_{i+1} + M^i(w - w_{i+1})) \text{ pour } w \geq w_{i+1}$$

Complexité: $O(nW)$.